

LLM deployment

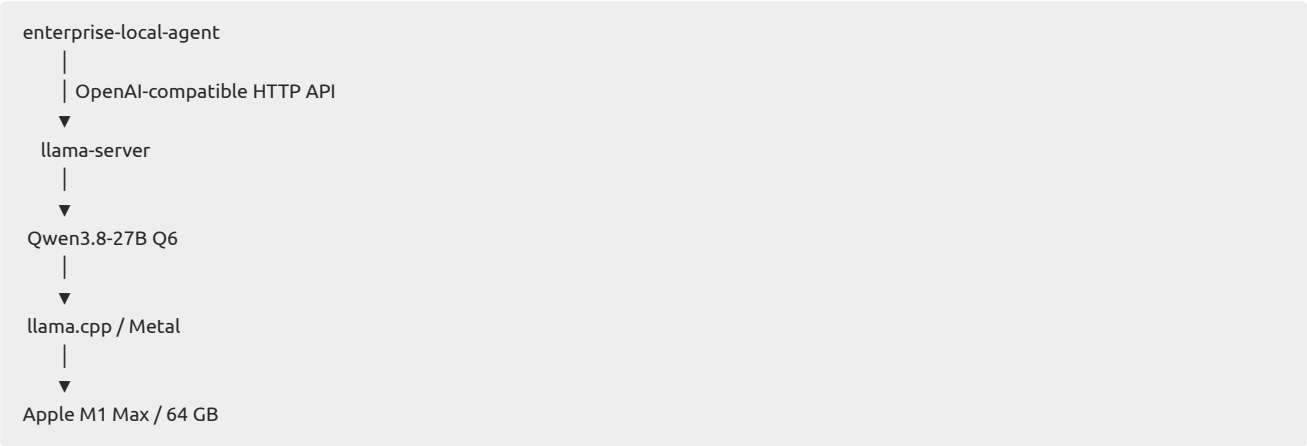
B0 Baseline Deployment Validation Report

Qwen3.8-27B Q6 on Apple M1 Max 64 GB

Baseline ID: B0-QWEN38-27B-Q6-M1MAX
Date: 2026-09-19
Status: VALIDATED
Scope: Single-session local inference
Concurrency: 1

1. Objective

Validate that the following local inference stack is technically viable as the first production-like LLM backend for the enterprise-local-agent :



The baseline focuses on:

- model feasibility on 64 GB unified memory;
- Metal inference stability;
- cold prompt-processing performance;
- autoregressive generation performance;
- long-context behaviour;
- llama-server LCP/KV prompt-cache effectiveness;
- suitability for later agent integration.

2. Hardware Platform

2.1 Machine

Component	Configuration
Machine	MacBook Pro
Model identifier	MacBookPro18,2
Architecture	ARM64
SoC	Apple M1 Max
CPU	10 cores
Performance cores	8
Efficiency cores	2

Component	Configuration
GPU	24 cores
Unified memory	64 GB LPDDR5
GPU API	Metal 4
SSD	~500 GB Apple NVMe
Free storage at baseline	~346 GiB

3. Software Platform

Component	Baseline
macOS	26.5.1
Architecture	Darwin ARM64
llama.cpp	0.4.1
llama.cpp build	10964
Commit	b29c606e2
Compiler used for binary	AppleClang 21
llama-server	/opt/homebrew/bin/llama-server
Installation	Homebrew
Ollama	Not installed
MLX	Not used
Backend	llama.cpp Metal

4. Model Baseline

Model family:	Qwen3.8-27B
Quantization:	UD-Q6_K_XL
Inference format:	GGUF
Inference engine:	llama.cpp
GPU acceleration:	Metal
GPU offload:	all available layers
Parallel slots:	1
Initial context:	32K
Prompt cache:	enabled / llama-server default behaviour
Jinja chat templates:	enabled
Metrics:	

enabled

Performance logging:
enabled

Baseline server profile:

```
llama-server \  
-hf unsloth/Qwen3.8-27B-GGUF:UD-Q6_K_XL \  
-host 127.0.0.1 \  
-port 8080 \  
-n-gpu-layers all \  
-ctx-size 32768 \  
-parallel 1 \  
-flash-attn on \  
-threads 8 \  
-threads-batch 8 \  
-batch-size 2048 \  
-ubatch-size 512 \  
-jinja \  
-metrics \  
-perf
```

5. Performance Test Overview

Three important scenarios were validated:

- A — Cold long-context request
~11.5K input tokens
no useful previous context
- B — Identical request
essentially complete LCP/KV reuse
- C — Same large prefix, changed suffix
realistic multi-turn-agent pattern

In addition, an earlier small-prompt measurement was retained as a short-context decode reference.

6. Short-Prompt Diagnostic

Initial `/metrics` measurement:

Prompt throughput:
19.7914 tokens/s

Generation throughput:
10.5323 tokens/s

Interpretation:

- 10.53 tok/s demonstrated good short-context decode capability.
- 19.79 tok/s was not representative of sustained prefill performance.
- the input was too small to efficiently utilize batched Metal prompt processing.

The later long-prompt benchmark confirmed this interpretation.

Therefore:

10.53 tok/s decode:
useful short-context reference

19.79 tok/s prefill:
NOT retained as long-context baseline

7. Test A — Cold 11.5K Context

7.1 Measured values

Prompt tokens: 11,459
Generated tokens: 256

Prompt eval time: 115.343 s
Generation time: 36.272 s
Total time: 151.615 s

Prompt throughput: 99.35 tok/s
Generation: 7.03 tok/s reported by llama-server

Truncated: no

Calculated directly from elapsed time:

11,459 / 115.343
= 99.347 tok/s

256 / 36.272
≈ 7.06 tok/s

For baseline comparison, retain the server-reported:

decode baseline = 7.03 tok/s

8. Cold Prefill Scaling

llama-server exposed intermediate prompt-processing progress:

Processed context	Average throughput
2,090 tokens	134.03 tok/s
4,138	117.55 tok/s
6,186	111.91 tok/s
8,234	108.56 tok/s
10,282	106.12 tok/s
10,943	101.72 tok/s
11,455	104.77 tok/s
Final 11,459	99.35 tok/s

Observed behaviour:

```
small context
~134 tok/s
|
▼
context grows
|
▼
~100 tok/s sustained at ~11.5K
```

There is no abrupt performance collapse.

9. Cold TTFT

The dominant cold-request latency comes from prompt evaluation.

Measured prefill:

```
115.343 s
```

Approximate time to first generated token adds roughly one decode-token latency:

```
~141 ms/token
```

Therefore:

```
Cold TTFT ≈ 115.5 s
```

This is the principal weakness of a fresh 11.5K context.

10. Test B — Identical Request / Maximum Prefix Reuse

llama-server:

```
selected slot by LCP similarity

f_sim_best = 1.000
f_keep    = 0.978
```

Actual prompt processing:

```
Prompt tokens reprocessed: 4

Prompt eval:
518.43 ms

Generation:
35,994.87 ms / 256 tokens
7.08 tok/s

Total:
36,513.31 ms
```

10.1 Derived Results

Prompt processing reduction

115.343 s
↓
0.518 s

Speedup:

$115.343 / 0.518$
 $\approx 222.5\times$

Tokens avoided

Cold:

11,459 tokens processed

Cached:

4 tokens processed

Approximate prompt reuse:

99.965%

Approximate TTFT

0.518 s prefill
+
~0.141 s first-token decode
≈
0.66 s TTFT

End-to-end latency

Cold:
151.62 s

Cached:
36.51 s

Overall request speedup:

$151.62 / 36.51$
 $\approx 4.15\times$

11. Test C — Same Prefix + Changed Final Instruction

This test is more representative of an actual multi-turn agent.

llama-server selected the existing slot using:

f_sim_best = 0.999
f_keep = 0.977

Measured values:

Prompt tokens reprocessed: 518

Prompt eval:
5.910 s

Prompt throughput:
87.65 tok/s

Generation:
36.027 s / 256 tokens
7.08 tok/s

Total:
41.937 s

12. Test C Derived Results

Prompt processing speedup

Cold:
115.343 s

Cached-prefix:
5.910 s

Speedup:

≈ 19.52×

Effective token reuse

Cold prompt:

11,459 tokens

Reprocessed:

518 tokens

Approximate prompt work avoided:

95.48%

This percentage is derived from actual reprocessed-token counts and should not be confused with llama-server's internal `f_keep` metric.

Approximate TTFT

5.910 s
+
~0.141 s first decode token
~
6.05 s

Overall latency

Cold:
151.62 s

Prefix reused:
41.94 s

Overall request speedup:

≈ 3.62×

13. Consolidated Performance Scorecard

KPI	Cold A	Cached B	Changed suffix C
Original context	~11.5K	~11.5K	~11.5K
Prompt tokens actually evaluated	11,459	4	518
Prompt processing	115.34 s	0.52 s	5.91 s
Prompt throughput	99.35 tok/s	7.72 tok/s*	87.65 tok/s
Approx TTFT	115.5 s	0.66 s	6.05 s
Generated tokens	256	256	256
Decode	7.03 tok/s	7.08 tok/s	7.08 tok/s
Generation time	36.27 s	35.99 s	36.03 s
Total latency	151.62 s	36.51 s	41.94 s
Prompt speedup vs cold	1×	222.5×	19.5×
Total speedup vs cold	1×	4.15×	3.62×
Prompt work avoided	0%	99.97%	95.48%
LCP similarity	—	1.000	0.999
Truncated	No	No	No

The 7.72 tok/s value for Test B is not meaningful as a prompt-throughput benchmark because only four prompt tokens were actually evaluated. The important measurement is the 518 ms total prompt-processing latency.

14. Decode Stability

Long-context generation remained extremely stable:

Test A:
~7.03 tok/s

Test B:
~7.08 tok/s

Test C:
~7.08 tok/s

Observed sustained long-context decode envelope:

~7.0–7.1 tokens/s

Short-context decode previously reached:

~10.53 tokens/s

Therefore the currently observed decode range is:

~7–10.5 tok/s

depending on active context length.

15. Generation-Latency Reference

Using the sustained long-context value of approximately 7.08 tok/s :

Output size	Approx generation time
32 tokens	~4.5 s
64	~9.0 s
128	~18.1 s
256	~36.2 s
512	~72.3 s
1,024	~144.6 s

This is important for agent design.

Internal agent decisions should preferably remain compact:

routing / decision:
~20–60 tokens

structured action:
~30–100 tokens

verification:
~30–100 tokens

final user answer:
workload dependent

A cached-prefix model call producing only 64 tokens could therefore realistically have a latency in roughly the:

~10–15 second

range when only a small prompt suffix must be evaluated.

16. Where Time Is Spent

Cold request

```
Total    151.6 s
|
├─ Prompt 115.3 s ≈ 76%
|
└─ Decode  36.3 s ≈ 24%
```

Identical cached request

```
Total    36.5 s
|
├─ Prompt   0.5 s ≈ 1.4%
|
└─ Decode  36.0 s ≈ 98.6%
```

Same-prefix changed-suffix

```
Total    41.9 s
|
├─ Prompt   5.9 s ≈ 14.1%
|
└─ Decode  36.0 s ≈ 85.9%
```

This is the most important baseline finding.

Once prompt reuse works, generation becomes the dominant latency, not context ingestion.

17. Prompt-Cache Validation

Prompt/KV reuse is considered:

VALIDATED

Observed:

```
Identical prompt:
  LCP similarity = 1.000
  only 4 tokens reevaluated

Nearly identical prompt:
  LCP similarity = 0.999
  only 518 tokens reevaluated
```

This establishes that llama-server can efficiently preserve expensive prefix computation across related requests.

For the future agent request builder, this creates an important architectural requirement:

```
Stable prefix
┌───────────┴───────────┐
system instructions
policy
tool definitions
project instructions
stable session context
└───────────┬───────────┘
```

Volatile suffix

conversation evolution
retrieved evidence
tool observations
current user input

Stable content should remain deterministic in:

- ordering;
- serialization;
- whitespace where possible;
- tool-definition order;
- policy representation;
- system-prompt representation.

Avoid inserting frequently changing timestamps, random IDs, runtime counters, or reordered metadata near the beginning of the prompt.

18. Hardware Resource Snapshot

At the hardware-report snapshot:

Physical memory:
64 GB

Snapshot:
~33 GB used
~31 GB unused

Swap:
~369 MB used / 2 GB configured

CPU:
~88% idle

This snapshot was not captured as a synchronized peak-memory measurement during the Qwen benchmark, so it must **not** be interpreted as Qwen's actual inference memory consumption.

A dedicated peak RSS / Metal-memory benchmark remains to be performed.

19. Thermal Status

The machine report recorded:

No thermal warning
No performance warning
No CPU power warning

However, this was not a dedicated sustained-load thermal benchmark.

Therefore:

Short-run stability:
validated

Sustained thermal behaviour:
not yet validated

20. Validation Result by Area

Area	Result
Model loads on 64 GB platform	✔ Validated
ARM64 llama.cpp runtime	✔ Validated
Metal inference	✔ Validated
Full Q6 27B local inference	✔ Validated
11.5K prompt without truncation	✔ Validated
Cold prefill	✔ ~99 tok/s
Long-context decode	✔ ~7 tok/s
Short-context decode	✔ ~10.5 tok/s observed
LCP slot selection	✔ Validated
KV/prefix reuse	✔ Validated
Identical-prefix reuse	✔ Excellent
Changed-suffix reuse	✔ Excellent
OpenAI-compatible HTTP endpoint	✔ Validated
32K maximum-context stress	⌚ Not tested
64K context	⌚ Not tested
128K context	⌚ Not tested
Q8 KV cache	⌚ Not tested
Q5 vs Q6 comparison	⌚ Not tested
Multiple concurrent slots	⌚ Not tested
Sustained thermal benchmark	⌚ Not tested
Peak unified-memory measurement	⌚ Not tested
Real agent workflow	⌚ Next phase

21. Baseline Performance Summary

MODEL
Qwen3.8-27B UD-Q6_K_XL

HARDWARE
Apple M1 Max
24-core GPU
8P + 2E CPU
64 GB unified LPDDR5

COLD ~11.5K CONTEXT
Prompt:
11,459 tokens
115.34 s
99.35 tok/s

Approx TTFT:
~115.5 s

Generation:
256 tokens
36.27 s

~7.03 tok/s

Total:

151.62 s

IDENTICAL CACHED CONTEXT

Prompt reevaluated:

4 tokens

Prompt latency:

0.518 s

Approx TTFT:

~0.66 s

Generation:

~7.08 tok/s

Total:

36.51 s

Prompt speedup:

~222.5×

Total speedup:

~4.15×

SAME PREFIX + CHANGED SUFFIX

Prompt reevaluated:

518 tokens

Prompt latency:

5.91 s

Prompt throughput:

87.65 tok/s

Approx TTFT:

~6.05 s

Generation:

~7.08 tok/s

Total:

41.94 s

Prompt speedup:

~19.5×

Total speedup:

~3.62×

Prompt work avoided:

~95.5%

SHORT-CONTEXT REFERENCE

Generation:

~10.53 tok/s

22. Baseline Conclusion

B0 is validated for single-session local agent inference.

The combination:

```
Apple M1 Max / 64 GB
+
Qwen3.8-27B UD-Q6_K_XL
+
llama.cpp / Metal
+
llama-server
```

provides sufficient performance for an initial local Enterprise Agent MVP.

Key observations:

1. **Q6 27B fits the platform and runs stably.**
2. **Cold large-context loading is expensive.** Approximately 115 seconds were required to process a fresh 11.5K-token context.
3. **Sustained cold prefill performance is good.** Approximately 99 tokens/s was achieved for the complete 11.5K context.
4. **Long-context generation is stable at approximately 7 tokens/s.**
5. **Small-context generation can reach approximately 10.5 tokens/s.**
6. **Prompt/KV reuse is extremely effective.** Identical requests reduced prompt processing from ~115 seconds to ~0.5 seconds.
7. **Realistic same-prefix/new-suffix requests reduced prompt processing to ~5.9 seconds.**
8. Once prefix reuse is active, **decode rather than prompt processing becomes the dominant model latency.**
9. The largest near-term performance opportunity for the Enterprise Agent is therefore not aggressive model quantization. It is:
 - deterministic prompt construction;
 - prefix stability;
 - effective KV reuse;
 - compact internal model outputs;
 - bounded retrieved context.
10. There is currently **no evidence requiring a downgrade from Q6 to Q5/Q4** for the first real-agent smoke test.

23. Frozen B0 Reference

For future comparisons, B0 should remain unchanged:

```
B0
├─ M1 Max / 24 GPU cores / 64 GB
├─ Qwen3.8-27B UD-Q6_K_XL
├─ llama.cpp 0.4.1 build 10964
├─ Metal
├─ context = 32K
├─ parallel = 1
├─ GPU layers = all
├─ Flash Attention = on
├─ threads = 8
├─ batch = 2048
├─ ubatch = 512
└─ KV/prompt cache = default llama-server behaviour
```

Recommended future baseline IDs:

```
B0 Q6 / 32K / default KV ← current frozen reference
B1 Q6 / 64K
B2 Q6 / 64K / Q8 KV
B3 Q5 / 32K
B4 Q6 / parallel=2
B5 real enterprise-agent E2E
```

All later experiments should change **one major dimension at a time** and compare against this B0 baseline.

24. Key Reference Metrics

The most important baseline numbers to retain are:

Cold prefill:	99.35 tok/s
Long-context decode:	~7.03 tok/s
Short-context decode:	~10.53 tok/s
Cold TTFT:	~115.5 s
Identical-prefix TTFT:	~0.66 s
Changed-suffix TTFT:	~6.05 s
Identical-prefix prompt speedup:	~222.5×
Changed-suffix prompt speedup:	~19.5×
Prompt work avoided:	~99.97% identical request ~95.48% changed suffix

These results establish deterministic prefix reuse as a major architectural performance lever for the local Enterprise Agent.